

# PromptWars: AI Placement Prep (working plan)

---

**Constraints:** 24-hour build · team of 3–4 · no mandated stack **Core loop:** diagnose → target → practice → re-measure **Differentiators:** Mistake Diagnosis · Project Defense · Communication Coach

---

## 1. Flow critique

### Problems to fix

1. **Onboarding asks too much before giving any value.** Six required fields plus an assessment will lose users (and judges) before they see anything. Keep only these as required: target role, target companies (max 3), and placement date. Take name and photo from Google. Default hours/day to 2 and edit it later. Drop "interests": it is vague and nothing downstream reads it.
2. **"Overconfidence" is not the same kind of thing as the other three types.** Concept, application and careless errors explain *why* an answer was wrong. Overconfidence describes *how sure the student was*. A student can make a concept gap *and* be overconfident. Model it as `primaryType` (concept | application | careless) plus an `overconfident: boolean` flag. The roadmap still reacts to both, and the data stops contradicting itself.
3. **Classification has to be explainable.** If an LLM labels every mistake with no rules behind it, judges can't verify the result and it costs a lot. Rules (Section 6, P5) classify first, using time taken, confidence, the misconception tag on the chosen distractor, and the student's history on that skill. The LLM only settles ambiguous cases and writes the explanation the student sees.
4. **Coding assessment needs code execution, and a sandbox won't fit in 24h.** Use code-reasoning MCQs instead: predict the output, find the bug, pick the complexity, pick the next line. They test DSA understanding without Judge0. Say this openly in the pitch.
5. **"Re-assessment" is undefined.** A full re-test after every session is annoying and slow. Define three update speeds:
  - *Per attempt:* skill mastery updates immediately, with no LLM call.
  - *Per session end:* the readiness score is recalculated and a snapshot is saved.
  - *On threshold:* the roadmap is regenerated only when a skill changes mastery band or a mistake pattern appears ( $\geq 3$  of the same type on one skill). At most once a day, plus a manual "Refresh plan" button.
6. **The Readiness Score needs a bar for each company.** "Readiness for Amazon" means nothing without Amazon's weights and cut-offs. Hand-author 5 company profiles as seed data (Section 5). Before the assessment, show "Not enough data", not 0%.
7. **Project Defense edge cases:**
  - No GitHub → fall back to a resume-only interview and mark the evidence as "unverified".
  - Forked, empty or huge repos → skip forks and read at most about 8 key files.
  - Private repos → out of scope.
  - GitHub rate limit (60/hr unauthenticated) → call from the server with a token.
8. **LinkedIn can't be read** (no API, and scraping breaks the ToS). Store only the URL. The LeetCode import uses an unofficial API, so it is stretch scope.

9. **Communication Coach:** handle denied mic permission (fall back to a typed answer, scored only on structure and relevance). Delete raw audio after transcription. Score only from the transcript and timing, so accent is never an input and can't be scored.
10. **Home with 8 modules is too busy.** Headline readiness, then one "Today" card with the single next action, then a module grid.
11. **Missing: a seeded demo account.** Judges won't sit through onboarding and a 15-minute assessment. Pre-seed a user with 2 weeks of history so the loop, trends and the re-record comparison are visible within 30 seconds.
12. **The Quick Assessment must be short and resumable.** About 12 minutes: 6 aptitude, 6 CS fundamentals, 6 code reasoning, 1 spoken answer. Each section can be skipped independently. Partial results give a partial score labelled "low confidence".

## 2. Screens

| #  | Screen             | Purpose             | Key UI elements                                                                                                                                           |
|----|--------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Login              | Entry point         | Google Sign-In button, one-line value prop, "Try demo account"                                                                                            |
| 2  | Onboarding stepper | Collect targets     | Steps: Target (role, companies) → Timeline (date, hours/day) → Optional (GitHub, LeetCode, LinkedIn, resume upload), with progress bar and "Skip for now" |
| 3  | Assessment intro   | Set expectations    | 4 section cards with time estimates, "Start" / "Skip, I'll do it later"                                                                                   |
| 4  | Assessment runner  | Measure             | Question, options, <b>confidence chips (Sure / Think so / Guessing) before submit</b> , section timer, save & exit                                        |
| 5  | Assessment results | First reveal        | Readiness score per company, top 3 gaps, mistake-type breakdown, "See my plan"                                                                            |
| 6  | Home               | Hub                 | Readiness per company (headline), Today card (next task), module grid, streak                                                                             |
| 7  | Readiness detail   | Transparency        | Per-component bars against the company bar, weights table, "biggest lever" callout, trend line                                                            |
| 8  | Skill Gap          | Where I stand       | Skill heatmap (category → skill), mistake-type distribution per skill                                                                                     |
| 9  | Mistake Journal    | Diagnose            | List of mistakes with type badge, "why we think so" signals, fix action, "mark understood"                                                                |
| 10 | Roadmap            | What and when       | Week view of tasks tagged with skill and targeted mistake type, rationale text, "Refresh plan"                                                            |
| 11 | Daily Practice     | Practise            | Same runner as #4; post-session summary with mastery deltas and new mistakes                                                                              |
| 12 | Projects           | Evidence            | Repo and resume project cards; detail view shows <b>claims table (claim → verdict → evidence file links)</b>                                              |
| 13 | Interview session  | Defend and practise | Mode (Project Defense / Technical / HR), chat-style Q&A, text or voice answer, end button                                                                 |
| 14 | Interview report   | Feedback            | Per-question scores, gaps, ideal answer outline, skills affected                                                                                          |

| #  | Screen              | Purpose      | Key UI elements                                                                                                                                            |
|----|---------------------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15 | Communication Coach | Speak better | Prompt → record → metrics (WPM, fillers, length) + structure score → <b>restructured version of their own answer</b> → re-record → side-by-side comparison |
| 16 | Progress            | Trend        | Readiness over time, mastery trend, practice minutes                                                                                                       |
| 17 | Profile             | Edit         | Targets, timeline, links, resume replace, delete data                                                                                                      |

### 3. Data model (Firestore)

Global seed collections are read-only for clients, except `questions`, which clients cannot read at all. Per-user data lives under `users/{uid}`; clients read only their own, and all writes go through API routes using the Admin SDK.

```

companies/{companyId}                                // seeded
  name, tier: "service"|"product"|"startup"
  weights: { aptitude, dsa, cs, projects, communication } // sum = 1
  bars:    { aptitude, dsa, cs, projects, communication } // 0-100 target
  skillImportance: { [skillId]: 1|2|3 }
  rounds: string[]

skills/{skillId}                                     // seeded taxonomy
  category: "aptitude"|"dsa"|"cs"|"communication"
  name, parentId?, prerequisites: skillId[]

questions/{questionId}                               // seeded bank (~150); SERVER-ONLY
  skillId, type: "mcq"|"code_output"|"code_bug"|"spoken"
  level: "concept"|"application", difficulty: 1-5, expectedTimeSec
  prompt, options[], correctIndex, explanation
  distractorTags: { [optionIndex]: "misconception:<id>"|"slip" }
  // Clients never read this collection. API routes return PublicQuestion:
  // Question minus correctIndex, explanation and distractorTags.

commPrompts/{promptId}                               // seeded (10: 4 behavioral, 4 technical, 2 opinion)
  text, kind: "behavioral"|"technical"|"opinion", targetDurationSec, skillId?

users/{uid}
  name, email, photoURL, createdAt, lastActiveAt
  onboardingStep: "target"|"timeline"|"optional"|"assessment"|"done"
  assessmentStatus: "none"|"in_progress"|"skipped"|"done"
  profile: { targetRole, targetCompanies: companyId[], placementDate,
             hoursPerDay, links: { github?, linkedin?, leetcode? },
             resume?: { storagePath, parsedAt } }
  readiness: { [companyId]: { score, components, confidence: "low"|"medium"|"high", updatedAt } } //
latest, denormalised
  activeRoadmapId?

users/{uid}/skillStates/{skillId}
  mastery: 0-1, attempts, correct, lastPracticedAt
  mistakeCounts: { concept, application, careless, overconfident }
  calibration: { sureCorrect, sureWrong } // for overconfidence feedback
  levelStats: { concept: { attempts, correct }, application: { attempts, correct } } // default 0;
read by P4

users/{uid}/assessments/{assessmentId}
  kind: "quick"|"recheck", status, startedAt, completedAt
  sections: { [section]: { status, questionIds[], score } }

users/{uid}/attempts/{attemptId}
  questionId, skillId, source: "assessment"|"practice"|"interview"

```

```

    sourceId, answer, isCorrect, confidence: "sure"|"likely"|"guess"|null // null when not asked
(e.g. interviews)
    timeTakenSec, createdAt, mistakeId?

users/{uid}/mistakes/{mistakeId}
    attemptId, skillId, primaryType: "concept"|"application"|"careless"
    overconfident: bool, classifiedBy: "rule"|"llm"
    signals: { timeRatio, confidence (nullable), distractorTag, skillMastery }
    explanation, fixAction, resolved, resolvedAt?

users/{uid}/roadmaps/{roadmapId}
    generatedAt, trigger, rationale, weeks: [{ week, focus, taskIds[] }]

users/{uid}/tasks/{taskId}
    roadmapId, skillId?, kind: "learn"|"drill"|"timed_drill"|"calibration"
        |"mock_interview"|"project_defense"|"comm_drill"
    targetsMistakeType?, title, estMinutes, dueDate, status, completedAt?

users/{uid}/projects/{projectId}
    source: "github"|"resume", repoUrl?, name, summary, stackDetected[]
    claims: [{ id, text, verdict: "supported"|"partial"|"unsupported"|"unverified",
        evidence, fileRefs[] }]
    interviewHooks[], analyzedAt
    defenseScore? // overall.score of the latest completed project_defense session for this project

users/{uid}/interviewSessions/{sessionId}
    mode: "project_defense"|"technical"|"hr", companyId?, projectId?
    turns: [{ question, intent, targetSkillId?, answer, eval? }] // ≤ 12 turns
    overall: { score, strengths[], gaps[] }, status, createdAt

users/{uid}/commAttempts/{attemptId}
    promptId, parentAttemptId? // links a re-record to the original
    transcript, durationSec, wpm, fillerCount, fillers: { [word]: n }
    scores: { structure, relevance, clarity, length } // 0-5
    overall // 0-100 = mean(scores) × 20, computed in code, never by the LLM
    structureDetected, missingParts[], restructured, tips[], createdAt

users/{uid}/readinessSnapshots/{snapshotId}
    createdAt, trigger, perCompany: { [companyId]: { score, components, confidence } }

```

**Relationships:** Attempt → Question → Skill. Mistake ↔ Attempt (1:1). Task → Skill and Roadmap. Project → InterviewSession (defense mode). CommAttempt → CommPrompt, and → parent CommAttempt (re-record chain). Snapshots give the history; `users.readiness` holds the latest values for a cheap Home read.

## 4. Module dependency map

| Module                  | Reads                                                     | Writes                                                                  | Triggered by                                                                                    |
|-------------------------|-----------------------------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Onboarding              | —                                                         | <code>users.profile</code> ,<br><code>onboardingStep</code>             | User                                                                                            |
| Assessment              | questions, skills                                         | attempts, mistakes, skillStates, assessments                            | User; completion → <b>Readiness + Roadmap</b>                                                   |
| Mastery updater (code)  | attempt, question, skillState                             | skillStates (mastery, levelStats, calibration)                          | Every attempt                                                                                   |
| Mistake classifier      | attempt, question, skillState                             | mistakes, skillStates.mistakeCounts                                     | End of session (batched)                                                                        |
| Readiness engine (code) | skillStates, projects, commAttempts, companies            | <code>users.readiness</code> ,<br>readinessSnapshots                    | End of any session, project analysis, comm attempt                                              |
| Roadmap generator       | profile, readiness gaps, skillStates, mistakes, companies | roadmaps, tasks                                                         | Assessment done; band change; $\geq 3$ same-type mistakes on one skill; manual (max 1/day auto) |
| Daily Practice          | tasks, skillStates, mistakes, questions                   | attempts, tasks.status (+ above chain)                                  | User from Today card                                                                            |
| Project analysis        | resume (Storage), GitHub API                              | projects                                                                | Resume upload / repo added                                                                      |
| Interview engine        | profile, projects, skillStates, companies                 | interviewSessions, projects.defenseScore, skillStates (verbal evidence) | User; end → Readiness                                                                           |
| Comm Coach              | commPrompts                                               | commAttempts (overall computed in code)                                 | User; save → Readiness                                                                          |

**Mechanism:** Next.js API routes call a `recompute(uid, trigger)` service synchronously at session end. There are no Cloud Function triggers: they are fewer moving parts and easier to debug in 24h.

**Mastery update rule (deterministic):**  $\text{mastery} \leftarrow \text{mastery} + \alpha \cdot (\text{outcome} - \text{mastery})$ , where  $\alpha = 0.15 + 0.05 \cdot \text{difficulty}$ . `outcome` is 1 if correct, 0 for a concept or application mistake, and **0.6 for a careless mistake**, because a slip is a consistency problem, not a knowledge gap.

## 5. Readiness Score

For company  $c$ , with components  $k \in \{\text{aptitude}, \text{dsa}, \text{cs}, \text{projects}, \text{communication}\}$ :

$$\text{Readiness}_c = \sum_k w_{ck} \cdot \min(1, S_k / B_{ck}) \cdot 100$$

- **$S_k$**  (student's component score, 0–100)

- aptitude / dsa / cs: mean of  $\text{mastery} \times 100$  across the category's skills, weighted by  $\text{skillImportance}$  for company c
  - projects: for each project,  $\text{projectScore} = 50 \cdot (\text{supported} + 0.5 \cdot \text{partial}) / \text{totalClaims} + 10 \cdot \text{defenseScore}$ , where  $\text{defenseScore}$  (0–5) is the overall score of the latest completed project\_defense session for that project, or 0 if there is none. **S\_projects = mean of the top 2 projectScores**; 0 with no projects.
  - communication: mean  $\text{overall}$  (0–100) of the last 3 comm attempts by  $\text{createdAt}$ .  $\text{overall} = \text{mean}(\text{structure}, \text{relevance}, \text{clarity}, \text{length}) \times 20$ , computed in code.
- **B\_ck**: the company's bar for that component. Reaching the bar gives full credit, and exceeding it gives no extra credit.
  - **w\_ck**: the company's weights (sum 1).
  - **Confidence** (low / medium / high), shown next to the score. Per component:

| Component           | Low                          | Medium                    | High                                                                            |
|---------------------|------------------------------|---------------------------|---------------------------------------------------------------------------------|
| aptitude / dsa / cs | < 6 attempts in the category | 6–9                       | $\geq 10$                                                                       |
| projects            | no analyzed project          | $\geq 1$ analyzed project | $\geq 1$ analyzed project <b>and</b> $\geq 1$ completed project_defense session |
| communication       | 0 attempts                   | 1–2                       | $\geq 3$                                                                        |

Overall confidence = the **lowest** level among components with weight  $\geq 0.15$  for that company.

- **Labels**:  $\geq 85$  Ready · 65–84 Close ·  $< 65$  Building.
- **Biggest lever** = the component with the largest  $w\_ck \cdot (1 - \min(1, S\_k/B\_ck)) \cdot 100$ . This is the number the roadmap prioritises.

**Seed profiles** (illustrative values; the team should refine them):

| Company    | w apt | w dsa | w cs | w proj | w comm | Bars (apt/dsa/cs/proj/comm) |
|------------|-------|-------|------|--------|--------|-----------------------------|
| TCS NQT    | .35   | .20   | .20  | .10    | .15    | 70/50/55/40/60              |
| Infosys    | .30   | .25   | .20  | .10    | .15    | 70/55/55/40/60              |
| Zoho       | .20   | .40   | .15  | .15    | .10    | 65/75/60/55/55              |
| Amazon SDE | .10   | .45   | .20  | .15    | .10    | 60/80/70/60/65              |
| Microsoft  | .05   | .45   | .20  | .15    | .15    | 60/85/70/65/70              |

**Worked example.** The student scores apt 63, dsa 48, cs 60, proj 30, comm 54.

- **TCS**:  $.35 \cdot 90 + .20 \cdot 96 + .20 \cdot 100 + .10 \cdot 75 + .15 \cdot 90 = \mathbf{91.7} \rightarrow \mathbf{Ready}$
- **Amazon**:  $.10 \cdot 100 + .45 \cdot 60 + .20 \cdot 85.7 + .15 \cdot 50 + .10 \cdot 83.1 = \mathbf{69.9} \rightarrow \mathbf{Close}$ . Biggest lever: DSA (18 of the 30 missing points).

The UI shows this table and arithmetic on screen #7, so a judge can recompute any score by hand.

## 6. AI prompt chain

Every call uses structured output (tool use with a JSON schema) and is validated with Zod on the server; on a validation failure it retries once. The model is `claude-sonnet-5` unless noted; high-volume calls use `claude-haiku-4-5-20251001`.

**P1. Resume parser** (Sonnet, PDF sent as a document block, no separate PDF library) Purpose: structure the resume and extract claims. In: resume PDF. Out:

```
{ "education":[{"institute":"","degree":"","cgpa":null}],
  "skills":[""],
  "projects":[{"name":"","description":"","repoUrl":null,
    "claims":[{"id":"c1","text":"","type":"tech|feature|metric|role","verifiable":true}]}],
  "experience":[{"org":"","role":"","claims":[]} ] }
```

**P2. Claim extractor**, used when a project comes from GitHub only (no resume). It is merged into P1 when a resume exists. In: README + repo description. Out: `{ "claims":[{"id","text","type","verifiable"}]}`

**P3. Repo analyzer** (Sonnet) Purpose: check claims against the code and find areas to probe in the interview. In: repo metadata, language stats, file tree (depth 3, truncated), README, manifests (package.json / requirements.txt), up to 8 key files chosen by heuristics (entry points, largest source files, routes, models), claims[]. Out:

```
{ "summary":"","stackDetected":[""], "architecture":"","complexity":1,
  "claimVerdicts":[{"claimId":"c1","verdict":"supported|partial|unsupported|unverified",
    "evidence":"","fileRefs":["src/app.py:40-72"]}],
  "redFlags":[""],
  "interviewHooks":[{"topic":"","why":"","fileRef":""}] }
```

**P4. Mistake classifier** (Haiku, one call per session with all wrong attempts batched) Purpose: settle ambiguous cases and write the explanation the student sees. Rules run first:

- `overconfident` = confidence is "sure" and the answer is wrong. Applies only when confidence is non-null; attempts with no confidence (e.g. interview answers) are never flagged.
- `careless` = `timeRatio` < 0.4, **or** (`mastery` ≥ 0.7 and the distractor tag is `slip`).
- `concept` = the distractor tag is a misconception **and** `mastery` < 0.5.
- `application` = the question level is application, **and** the student has ≥ 70% accuracy on concept-level questions for that skill (from `skillStates.levelStats.concept`).
- Anything else is ambiguous and goes to the LLM.

In: per item, the question, correct answer, chosen answer + distractor tag, confidence (or null), `timeRatio`, skill mastery, concept vs application accuracy, and the rule verdict (or null). Out:

```
{ "items":[{"attemptId":"","primaryType":"concept|application|careless",
"overconfident":false, "agreesWithRule":true,
"explanation":"<=2 sentences, second person",
"conceptToRevisit":"","fixAction":"" }]} }
```

**P5. Roadmap generator** (Sonnet; a hybrid approach) Code computes the task budget:  $\text{priority} = \text{companyWeight} \times \text{gap} \times (1 + \text{mistakeSeverity})$ , with minutes allocated from `hoursPerDay` and days left. The LLM only sequences the tasks and writes the text. Each mistake type maps to a task kind:

- concept → learn + basic drill
- application → worked example + mixed drill
- careless → timed accuracy drill
- overconfident → calibration drill (the student must justify the answer)

In: profile, days left, budget per skill, mistake distribution, open tasks. Out:

```
{ "rationale":"","weeks":[{"week":1,"focus":"","
"tasks":[{"skillId":"","kind":"","targetsMistakeType":"","title":"","estMinutes":30,"day":1}]}]} }
```

The server rejects the result if any `skillId` is unknown or a day's total exceeds  $\text{hoursPerDay} \cdot 60$ .

**P6. Interviewer** (Sonnet, one call per turn) Purpose: ask the next question and adapt to the student's answers. In: mode, company rounds, profile, weak skills; in defense mode, also the project summary, `interviewHooks` and unsupported claims; conversation so far. Out:

```
{ "question":"","intent":"probe_claim|depth|tradeoff|behavioral|concept",
"targetSkillId":null, "claimId":null, "followUpOf":null, "endInterview":false }
```

Defense mode asks about unsupported claims first ("Your resume says you built X. Walk me through where that lives in the code.").

**P7. Answer evaluator** (Sonnet, one call at the end of the session over all turns, which keeps each turn fast) Out:

```
{ "turns":[{"index":0, "scores":{"correctness":0,"depth":0,"clarity":0},
"strengths":[""], "gaps":[""], "idealOutline":[""],
"skillEvidence":[{"skillId":"","signal":"positive|negative"}],
"claimVerdictUpdate":{"claimId":null,"verdict":null} ]},
"overall":{"score":0,"strengths":[""], "gaps":[""]} }
```

**P8. Communication scorer** (Sonnet) Code computes WPM, filler counts, duration and pauses from the STT word timestamps before the call. The LLM never sees audio, only the transcript and numbers. In: prompt, question kind (behavioral → STAR; technical/opinion → claim-reason-example), transcript, metrics. Out:

```
{ "structureDetected":"STAR|CRE|none", "scores":{"structure":0,"relevance":0,"clarity":0,"length":0},  
  "missingParts":["Result"], "restructured":"","tips":["","",""] }
```

The LLM does not produce `overall`; code computes it from `scores` before saving the attempt. Hard rule in the prompt: `restructured` may only reuse the student's own content and must use placeholders like `[add a measurable result]` for missing facts. No comments on accent or pronunciation.

**P9. Re-record comparison** (Haiku; nice-to-have) In: both attempts' scores and transcripts. Out: `{ "improved":[""], "stillMissing":[""], "summary":"" }`

The Quick Assessment and practice questions come from the **seeded bank**, not live generation. That keeps them reliable and gradeable. The team can use an LLM before the event to draft about 150 questions, and **must check every answer key by hand**.

## 7. Tech stack

| Choice                                           | Why                                                                                                  |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Next.js (App Router) + TypeScript                | UI and API routes in one repo and one deploy                                                         |
| Tailwind + shadcn/ui                             | A polished UI with no design time                                                                    |
| Firebase Auth (Google)                           | Google Sign-In is already decided and takes minutes to set up                                        |
| Firestore                                        | Per-user documents match the data shape; realtime Home updates for free                              |
| Firebase Storage                                 | Resume PDFs and temporary audio                                                                      |
| Claude Sonnet 5 / Haiku 4.5 via Anthropic SDK    | Strong structured JSON, native PDF input, and long context for repo files                            |
| Zod                                              | One schema for LLM output validation and TS types                                                    |
| Deepgram (Nova, <code>filler_words=true</code> ) | Keeps "um/uh" in the transcript and returns word timestamps for pace; most STT engines strip fillers |
| MediaRecorder API                                | Browser audio capture with no extra libraries                                                        |
| Octokit + server GitHub token                    | 5,000 requests/hr and simple tree and file reads                                                     |
| Recharts                                         | Readiness trend and component bars                                                                   |
| Vercel                                           | Zero-config Next.js deploys with preview URLs for each teammate                                      |

Put all LLM calls behind one `llm.call(promptId, input, schema)` wrapper. If the judges favour a particular provider (PromptWars is Google-run), switching to Gemini touches only that one file.

## 8. Scope

### MVP (must work live in the demo)

- Google login, 3-state routing, and the reduced onboarding

- Quick Assessment: aptitude, CS and code-reasoning MCQs with confidence chips, plus 1 spoken answer
- Mistake Diagnosis (rules + P4), Mistake Journal
- Readiness Score for 5 seeded companies, with the transparent breakdown screen
- Roadmap (hybrid P5), Daily Practice, and a closed loop: practice → mastery → readiness → roadmap refresh
- **Project Defense:** resume upload + 1 GitHub repo → claims table → 5–6 question text interview → report
- **Communication Coach:** record → metrics → score → restructure → re-record → comparison
- Seeded demo account with history

### Nice-to-have

- Technical and HR mock interview modes (same engine, different prompt)
- Voice answers inside interviews (reusing the Comm Coach STT)
- Progress screen trends, P9 comparison summary
- Multiple repos, LeetCode stats import

### Stretch

- Live code execution (Judge0)
- LLM-generated question variants with auto-verification
- Spoken interviewer (TTS), spaced-repetition reminders
- College/TPO cohort dashboard

---

## 9. Build order (4 people; with 3, merge Track A into B)

### Hours 0–2: everyone

- Scaffold the repo, create the Firebase project, deploy "hello" to Vercel
- Write shared `types.ts` + Zod schemas from Section 3; the `llm.call` wrapper
- Freeze the skill taxonomy (~25 skills) and the 5 company profiles

### Hours 2–8: parallel tracks

- **A: Shell.** Auth, 3-state routing, onboarding stepper, Home layout with mock data, Readiness detail screen.
- **B: Engine.** Seed ~150 questions (checked by hand), assessment runner with confidence chips, attempts + mastery updater, readiness engine (a pure function with unit tests on the worked example).
- **C: Project Defense.** P1 resume parser, GitHub fetch + key-file picker, P3 analyzer, claims table UI.
- **D: Comm Coach.** Recorder, Deepgram integration, metrics, P8 scorer, feedback UI.

**Checkpoint 1 (hour 8):** a new user logs in, onboards, takes the assessment and sees a real readiness score on Home.

### Hours 8–14

- **B:** P4 mistake classifier + Journal, P5 roadmap + tasks, Daily Practice with the recompute chain.

- **C:** P6 interviewer + P7 evaluator in defense mode, report screen, write-back to projects and readiness.
- **D:** Re-record flow + comparison view; the spoken assessment question reuses the same component.
- **A:** Wire Home to live data, Today card, Skill Gap heatmap.

**Checkpoint 2 (hour 14):** loop closed. A practice session visibly changes mastery, readiness and the roadmap.

**Hours 14–18:** nice-to-haves only if the checkpoints passed (technical interview mode first). Also error and empty states, mic-denied fallback, and a no-GitHub fallback.

**Hour 18: feature freeze.**

**Hours 18–22:** seed the demo account (2 weeks of attempts, snapshots, a before/after comm attempt, an analyzed repo with 1 unsupported claim), finish the deploy, and fix bugs.

**Hours 22–24:** 3-minute demo script, rehearse twice, record a backup video in case the live demo fails.

**Demo order:** Amazon readiness 70 "Close" → tap it and show the arithmetic → one mistake shown as "careless, not concept" → the roadmap reacting → Project Defense catching a resume claim the code doesn't support → a comm answer re-recorded with a higher score → finish a short practice session live and watch the score move.

## Changelog

### 2026-09-26: Pre-split schema decisions (approved)

- Readiness confidence now has three levels (low / medium / high) with per-component thresholds; overall = lowest among components weighted  $\geq 0.15$  (Sections 3, 5).
- `skillStates.levelStats` added (concept / application attempts and correct, default 0) so P4's application rule can run (Sections 3, 6).
- `questions` is server-only; clients receive `PublicQuestion` (no correctIndex, explanation or distractorTags) via API routes (Section 3).
- `commPrompts` collection defined and seeded with 10 prompts (Section 3).
- `commAttempts` gains `overall` (0–100, computed in code) and a required `createdAt` (Sections 3, 5, 6).
- Projects component: per-project score uses the latest completed defense session; `S_projects` = mean of the top 2 (Section 5).
- `attempts.confidence` is nullable; P4's overconfident rule applies only when it is non-null (Sections 3, 6).

### 2026-09-26: Integration (all tracks built in one pass)

- Quick Assessment is 10 aptitude + 10 coding + 10 communication (`commPrompts`); CS fundamentals come from Mock Interview round 1 and Daily Practice.
- `questions` gains `pool` ("assessment" | "mock" | "practice") and `reviewed` (false until a human checks the key). 60 questions + 2 `codingProblems` seeded.

- New collections under `users/{uid} : categoryScores` (unweighted history for Progress), `skillGapReports` (P-GAP), `mockInterviews`.
- New prompts: P-GAP (skill-gap suggestions) and P-CODE (AI code review, not execution).
- Onboarding is 3 steps (target, timeline, assessment); links and resume moved to `/connect`.
- New skill states start at mastery 0.5 (the plan didn't specify an initial value).

## 2026-09-26: Gemini provider + domain coding patterns

- `LLM_PROVIDER=gemini` (REST API, JSON-schema output, retries on 429/5xx, fallback model). Free-tier keys have no Pro quota, so both tiers use Flash models.
- Questions gain optional `pattern`, `approach` and `domains` ("sde" | "fullstack" | "data-ml" | "service"); 32 "which approach?" questions, 8 per domain. The domain comes from the target role (keywords), or "service" when every target company is a service company.
- Daily Practice = 4 domain pattern questions + 6 weak-spot questions; the pattern and approach are revealed only after grading.

## 2026-09-26: Daily coding

- `practiceProblems`: 35 classics (29 LeetCode, 3 Codeforces, 3 HackerRank), restated in our own words, each with `source.url` to the original. Expected outputs come from reference solutions run at seed time; references never leave the seed. Tests are stored as a JSON string (Firestore can't nest arrays).
- Daily set: 3 problems per day for the user's domain (1 Easy + 2 Medium; Service domain 2 Easy + 1 Medium), unsolved and weakest-skill first, stable for the day.
- Code runs in the browser: JavaScript in a Web Worker, Python on Pyodide (from jsDelivr), with a 6 s limit that kills the worker. Submit runs all tests including hidden ones, then P-CODE reviews the code. C++/Java: AI review only. Submissions ( `users/{uid}/codingSubmissions` ) update skill mastery; the first solve records a readiness snapshot.
- Known trade-off: the browser reports the test results, so a determined user could fake a pass. Acceptable for self-practice; server-side execution (Judge0) would close it.